

混合推荐系统设计

实验原理

协同过滤

协同过滤是一种推荐算法。它可以基于用户相似性，也可以基于产品相似性。

基于用户相似性时，它通过收集众多用户的信息，发现用户之间的相似性，从而通过兴趣相似的用户所使用的产品，为某一用户进行推荐。基于产品相似性时，它通过分析用户的行为计算物品之间的相似度，从而为用户推荐相似的物品。

基于内容的推荐

基于内容的推荐也是一种推荐算法。它根据产品的特点，发现产品之间的相似性，并将与用户先前的正反馈产品相似的产品推荐给用户。例如，若用户A喜欢具有特征D的物品，那么系统就会推荐更多具有特征D的物品给用户A。

召回率

召回率是指真实为正的样本中，被模型正确分类为正的比例。在这里，就是用户喜欢的电影中，被推荐的比例。

实验过程

数据获取与处理

```
movie = pd.read_csv( filepath_or_buffer: 'movies.dat', sep = '::', header=None, names=['movie_id','title','genres'], engine='python', encoding= 'latin1')
rating = pd.read_csv( filepath_or_buffer: 'ratings.dat', sep = '::', header=None, names=['user_id','movie_id','rating','timestamp'], engine='python', en
user = pd.read_csv( filepath_or_buffer: 'users.dat', sep = '::', header=None, names=['user_id','gender','age','occupation','zip'], engine='python', enco
print(movie.head())
print(rating.head())
print(user.head())
```

首先，下载MovieLens 1M数据集并读取数据。结果如下：

movie_id		title		genres	
0	1	Toy Story (1995)	Animation Children's Comedy		
1	2	Jumanji (1995)	Adventure Children's Fantasy		
2	3	Grumpier Old Men (1995)	Comedy Romance		
3	4	Waiting to Exhale (1995)	Comedy Drama		
4	5	Father of the Bride Part II (1995)	Comedy		
user_id		movie_id	rating	timestamp	
0	1	1193	5	978300760	
1	1	661	3	978302109	
2	1	914	3	978301968	
3	1	3408	4	978300275	
4	1	2355	5	978824291	
user_id		gender	age	occupation	zip
0	1	F	1	10	48067
1	2	M	56	16	70072
2	3	M	25	15	55117
3	4	M	45	7	02460
4	5	M	25	20	55455

协同过滤实现（用户相似度）

```
# 协同过滤
user_rating = rating.pivot_table(index='user_id', columns='movie_id', values='rating')
# print(user_rating.head())
user_similarity = cosine_similarity(user_rating.fillna(0))
similarity_df = pd.DataFrame(user_similarity, index=user_rating.index, columns=user_rating.index)
```

计算用户-电影评分矩阵，使用余弦相似度计算用户之间的相似度，并将计算结果转化为Dataframe格式，方便查看。

```
users = similarity_df[user_id].sort_values(ascending=False).index[1:6]
rated = user_rating.loc[user_id].index
similar_ratings = user_rating.loc[users, rated]
recommend = similar_ratings.mean(axis=0).sort_values(ascending=False).head(20).index.tolist()
```

设置要进行推荐的用户id，获取与这个用户最相近的5个用户，那5个相似用户对所有电影的评分。最后，通过计算这5个用户对这些电影评分的平均值，按照平均分为这个用户推荐电影。

基于内容的推荐实现

```
binarizer = MultiLabelBinarizer()
movie_genre = binarizer.fit_transform(movie['genres'].str.split('|'))
movie_genre_df = pd.DataFrame(movie_genre, columns=binarizer.classes_, index=movie['movie_id'])
# print(movie_genre_df.head())
```

首先，将电影与类别之间的关系表转换为一个二进制矩阵。矩阵每一行表示一部电影，0或1表示这部电影是否属于这个类别。

```
movie_similarity = cosine_similarity(movie_genre_df)
movie_similarity_df = pd.DataFrame(movie_similarity, columns=movie['movie_id'], index=movie['movie_id'])
# print(movie_similarity_df.head())
```

计算电影之间的相似度。

```
user_ratings = rating[rating['user_id'] == 1]
movie_like = user_ratings[user_ratings['rating'] >= 4]['movie_id']
# print(movie_like)
movie_id = movie_like.values.tolist()[13]
print("movieid:", movie_id)
similar_movie = movie_similarity_df[movie_id].sort_values(ascending=False)
recommend_movie = similar_movie.head(30).index.tolist()
```

获取用户的评分和用户喜欢（评分大于4）的电影，在这个列表中随机选择一部电影。寻找和这部电影相似的电影列表，并推荐给用户。

实验结果

协同过滤（用户相似度）

设置用户评分大于4的电影为“用户喜欢的电影”，那么推荐结果与用户喜欢的电影的交集和召回率如下：

```
交集: {527, 2804, 1287}
召回率: 0.17
```

基于内容的推荐

推荐结果与用户喜欢的电影的交集和召回率如下：

```
交集:  {1, 2355}  
召回率: 0.11
```

两者召回率类似，协同过滤方法召回率更高一些。